

Experiment 1

Aim:

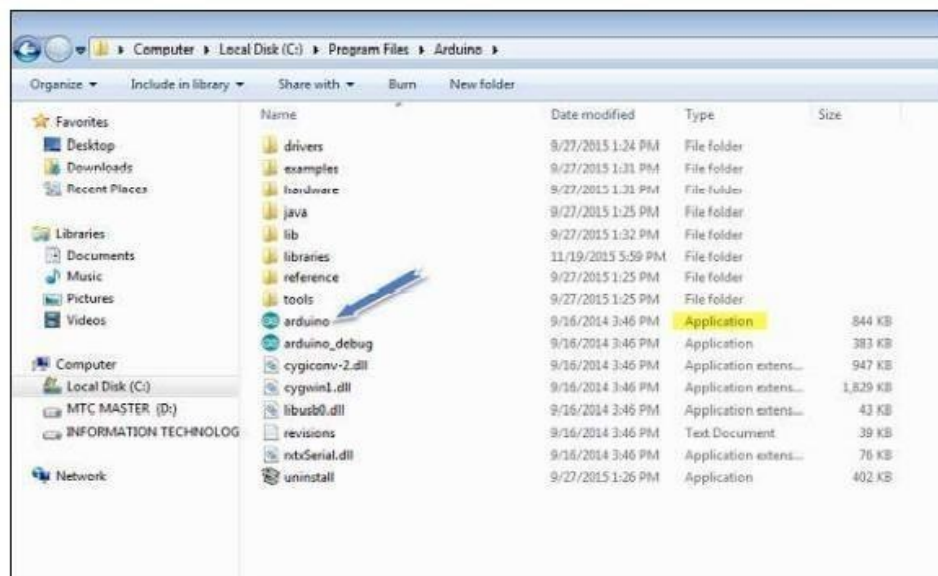
To familiarize with Arduino and perform the necessary software installation.

Description:

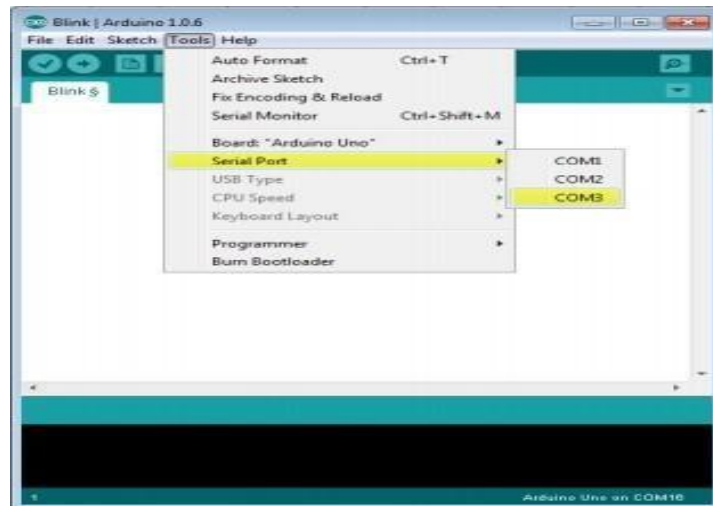
In this experiment, I became familiar with the Arduino platform and its development environment. I learned the basic features of Arduino and gained an introductory understanding of Raspberry Pi as an IoT platform. I installed and configured the Arduino IDE, explored its interface and built-in examples, and executed the **Blink** program. This experiment helped me understand the basic process of selecting a board, configuring the communication port, compiling a program, and uploading it to an Arduino board.

Procedure:

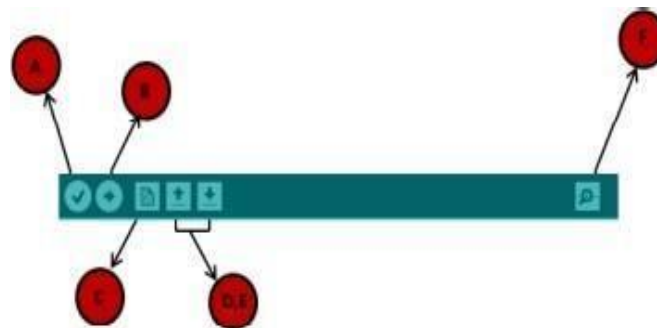
1. Downloaded the Arduino IDE software from the official website
[“https://www.arduino.cc”](https://www.arduino.cc).
2. I launched the Arduino IDE on my system.
3. After downloading the software, I extracted the folder and located the Arduino application file. I double-clicked the application icon to open the Arduino IDE.



4. Once the IDE was opened, I either created a new sketch or opened an existing example. I selected the **Blink** example by navigating to **File** → **Examples** → **01.Basics** → **Blink**. The program was used to make the onboard LED turn ON and OFF at regular time intervals.

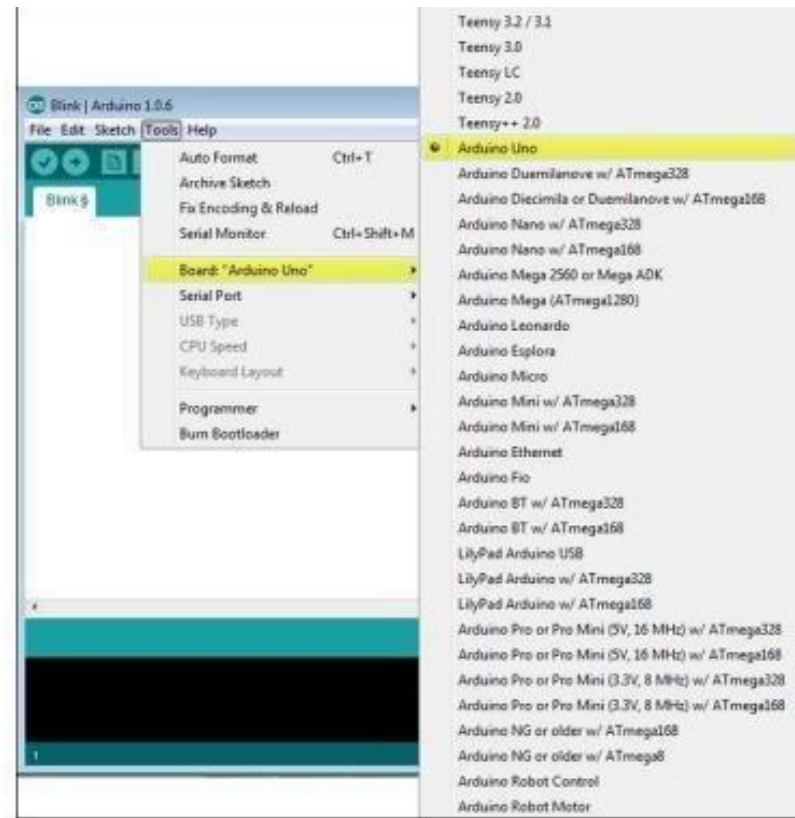


5. I selected the appropriate Arduino board by navigating to **Tools** → **Board** and choosing **Arduino Uno**, which matched the board I was using.
6. I selected the appropriate serial port by navigating to **Tools** → **Port**. I verified the correct COM port by disconnecting and reconnecting the Arduino board and identifying the port associated with it.
7. I became familiar with the Arduino IDE toolbar:



- **Verify** – Used to compile the program and check for errors.
- **Upload** – Used to upload the program to the Arduino board.
- **New Sketch** – Used to create a new program.
- **Open** – Used to open an existing sketch or example.
- **Save** – Used to save the current sketch.
- **Serial Monitor** – Used to communicate with the Arduino through the serial interface.

8. I clicked the **Upload** button to compile and upload the Blink program to the Arduino Uno board.



9. I observed the **RX and TX LEDs** flashing on the board during the uploading process. After a few seconds, the message **“Done uploading”** appeared in the status bar, indicating that the program had been uploaded successfully.

Result:

The Arduino IDE was successfully installed and configured. The **Arduino Uno** board and the appropriate serial port were selected, and the **Blink** program was successfully uploaded to the board. The onboard LED blinked at regular time intervals, confirming that the program was executed successfully.

Conclusion:

Through this experiment, I gained a basic understanding of the Arduino IDE, board configuration, serial port selection, and program uploading. I successfully executed the Blink program on the Arduino Uno, providing a foundation for further IoT and embedded-system applications.

Experiment 2

Aim:

To interface an LED/Buzzer with Arduino and write a program to turn ON the LED for 1 second after every 2 seconds.

Requirements:

- Standard 5mm LED
- One Resistor
- Arduino Uno microcontroller board (ATmega328p)
- Jumper wires

Description:

In this experiment, I interfaced an LED with the Arduino Uno microcontroller using a digital output pin. The LED was connected through a resistor to limit the current and protect the LED. I wrote and executed an Arduino program to turn the LED ON for 1 second and keep it OFF for 2 seconds continuously. This experiment helped me understand how Arduino digital pins can be used to control external output devices in IoT and embedded-system applications.

Procedure:

1. I accessed the **Wokwi online simulation platform** through a web browser and created a new simulation using the **Arduino Uno** board.
2. I added the necessary components, namely the **5 mm LED, resistor, jumper wires, and Arduino Uno**, to the simulation workspace.
3. I established the circuit connections by connecting the **anode of the LED to digital pin 3** through the resistor and connecting its **cathode to the Arduino GND**.
4. I entered the Arduino program in the **sketch.ino** file. The program configured digital pin 3 as an output and used appropriate digital states and delay intervals to control the LED.

Code:

/*Experiment: Interfacing an LED with Arduino Uno

Purpose: To turn ON the LED for 1 second and OFF for 2 seconds

Name: K.Y.TILAK REDDY

RollNo:25M11MC077 */

```
void setup() {  
  
  pinMode(3, OUTPUT);  
  
}  
  
void loop() {  
  
  digitalWrite(3, HIGH);  
  
  delay(1000);  
  
  digitalWrite(3, LOW);  
  
  delay(2000);  
  
}
```

5. I compiled the program and uploaded it to the Arduino Uno within the Wokwi simulation environment.
6. I executed the simulation and monitored the LED. The LED remained **ON for 1 second and OFF for 2 seconds**, and this sequence continued repeatedly.

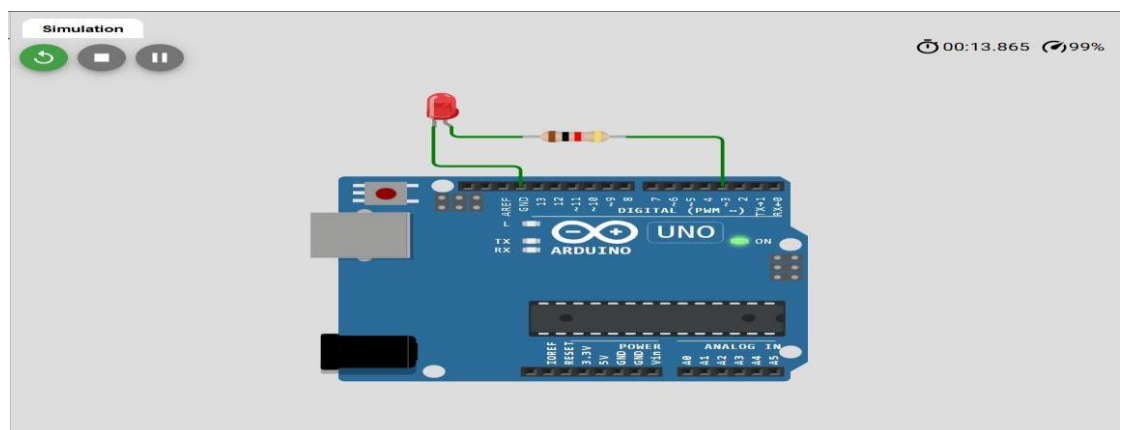
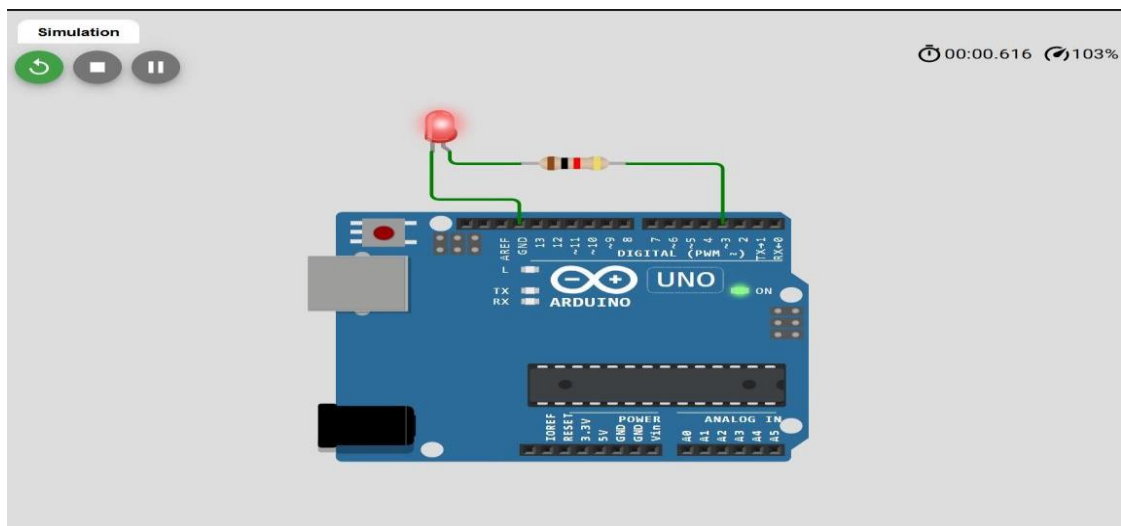
Result:

The LED was successfully interfaced with the Arduino Uno and operated according to the programmed timing sequence. The simulation confirmed that the LED switched ON for 1 second and OFF for 2 seconds continuously.

Before:



After:



Conclusion:

The experiment provided practical knowledge of connecting an external output device to an Arduino Uno and controlling it through a digital pin. I also gained an understanding of the functions **pinMode()**, **digitalWrite()**, and **delay()**, which are used for pin configuration, output control, and timing operations, respectively.

Experiment 3

Aim:

To interface a push button/digital sensor (IR/LDR) with Arduino and develop a program to turn ON the LED when the push button is pressed or when sensor detection occurs.

Requirements:

- Standard 5mm LED
- Two Resistors
- Arduino Uno microcontroller board (ATmega328p)
- Jumper wires
- Push button

Description:

This experiment focused on interfacing a push button as a digital input with the Arduino Uno and using its input state to control an LED. The push button was connected to a digital input pin, while the LED was connected to a digital output pin. The Arduino program was designed to detect the button press and change the state of the LED accordingly. This experiment provided practical knowledge of reading digital inputs and controlling output devices using conditional logic.

Procedure:

1. I accessed the **Wokwi online simulation platform** through a web browser and created a new simulation using the **Arduino Uno** board.
2. I added the required components, including the **LED, two resistors, jumper wires, push button, and Arduino Uno**, to the simulation workspace.
3. I connected the **anode of the LED to digital pin 3** of the Arduino through **resistor 1** and connected the **cathode of the LED to GND**.
4. I supplied **5V power** to pin **1:1L** of the push button.
5. I connected pin **1:1R** of the push button to **GND through resistor 2** and also connected it to **digital pin 5** of the Arduino Uno.
6. I entered the required Arduino program in the **sketch.ino** file. The program was used to read the push button state through digital pin 5 and control the LED connected to digital pin 3.

Code:

/* Experiment 3: Interfacing a Push Button with Arduino Uno

Purpose: To turn ON the LED when the push button is pressed

Name:K.Y.TILAK REDDY

RollNo:25M11MC077 */

```
int flag = 0;

void setup() {

pinMode(3, OUTPUT); // LED connected to pin 3

pinMode(5, INPUT); // Push button connected to pin 5

}

void loop() {

int value = digitalRead(5); // Read push button state

if (value == 1 && flag == 0) {

digitalWrite(3, HIGH); // Turn LED ON

delay(1000);

flag = 1; }

else if (value == 1 && flag == 1) {

digitalWrite(3, LOW); // Turn LED OFF

delay(1000);

flag = 0;

}

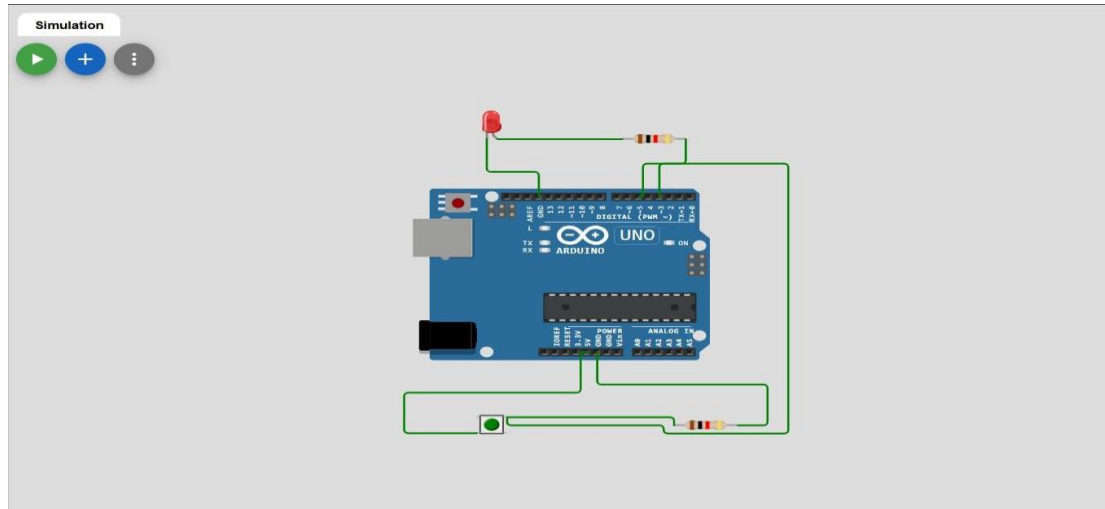
}
```

7. I compiled the program and uploaded it to the Arduino Uno within the Wokwi simulation environment.
8. I tested the circuit by pressing the push button and observed the LED response. The LED turned ON when the button was pressed and turned OFF when the button was pressed again.

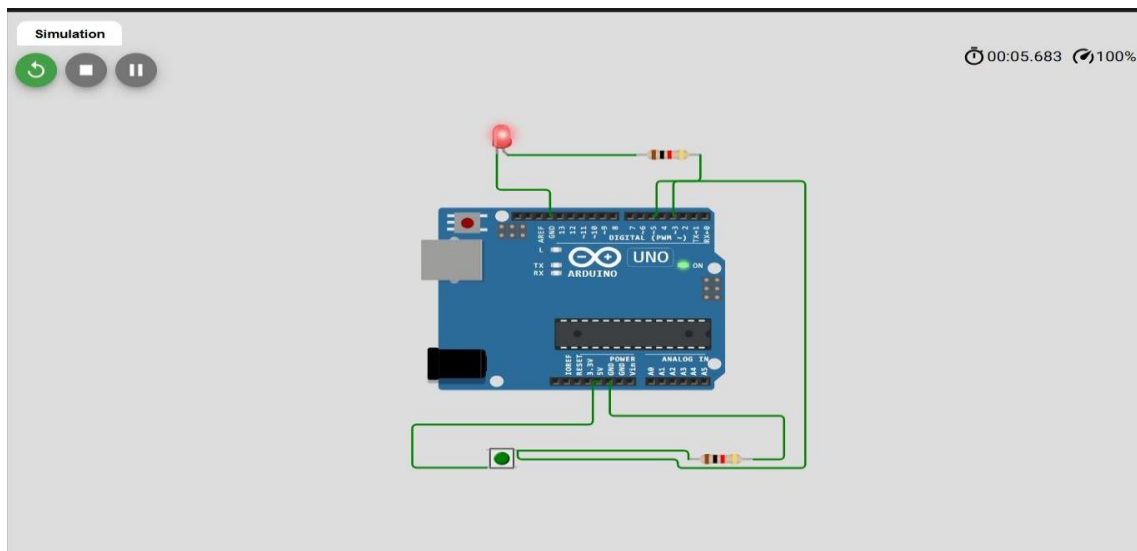
Result:

The push button was successfully interfaced with the Arduino Uno and used as a digital input to control the LED. The LED switched ON with the first button press and switched OFF with the subsequent press, confirming the successful execution of the program.

Before:



After:



Conclusion:

This experiment provided practical knowledge of interfacing a digital input device with the Arduino Uno. I learned how to read the state of a push button using **digitalRead()** and control an LED using **digitalWrite()**. I also understood how conditional statements and variables can be used to implement state-based control in Arduino and IoT applications.

Experiment 4

Aim:

To interface a DHT22 sensor with Arduino Uno and develop a program to measure and display temperature and humidity readings.

Requirements:

- DHT22 sensor
- Arduino Uno microcontroller board (ATmega328p)
- Jumper wires

Description:

This experiment involved connecting a DHT22 digital sensor to an Arduino Uno for monitoring environmental temperature and humidity. The sensor was configured to provide temperature and relative humidity data, which was retrieved by the Arduino and displayed through the Serial Monitor. The experiment provided practical knowledge of interfacing digital sensors, using sensor libraries, and acquiring environmental data for IoT applications.

Procedure:

1. I accessed the **Wokwi online simulation platform** using a web browser and selected the **Arduino Uno** board for the experiment.
2. I added the required components, including the **DHT22 sensor, jumper wires, and Arduino Uno**, to the simulation workspace.
3. I established the circuit connections using jumper wires as follows:
 - I connected the **VCC pin of the DHT22** to the **5V supply** of the Arduino Uno.
 - I connected the **SDA/Data pin of the DHT22** to **digital pin 3** of the Arduino Uno.
 - I connected the **GND pin of the DHT22** to the **GND** of the Arduino Uno.
4. I entered the required Arduino program in the **sketch.ino** file. The program initialized the DHT22 sensor, read the temperature and humidity values, and sent the readings to the Serial Monitor.

Code:

/* Experiment 4: Interfacing DHT22 Sensor with Arduino Uno

Purpose: To read and display temperature and humidity values

Name:K.Y.TILAK REDDY

Roll No: 25M11MC077*/

```
#include <DHT.h>
```

```
#define DHTPIN 3
```

```
#define DHTTYPE DHT22
```

```
DHT Sensor(DHTPIN, DHTTYPE);
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    Sensor.begin();
```

```
    // Display student details before starting readings
```

```
    Serial.println("-----");
```

```
    Serial.println("    Experiment 4: DHT22 Sensor Interface");
```

```
    Serial.println("    Name : K.Y.TILAK REDDY");
```

```
    Serial.println("    Roll No : 25M11MC077");
```

```
    Serial.println("-----");
```

```
    delay(2000);
```

```
}
```

```
void loop() {
```

```
    float tempr = Sensor.readTemperature();
```

```
    float humid = Sensor.readHumidity();
```

```
    // Check if sensor reading failed
```

```
    if (isnan(tempr) || isnan(humid)) {
```

```
        Serial.println("Failed to read from DHT22 sensor!");
```

```
        delay(2000);
```

```
        return;
```

```
    }
```

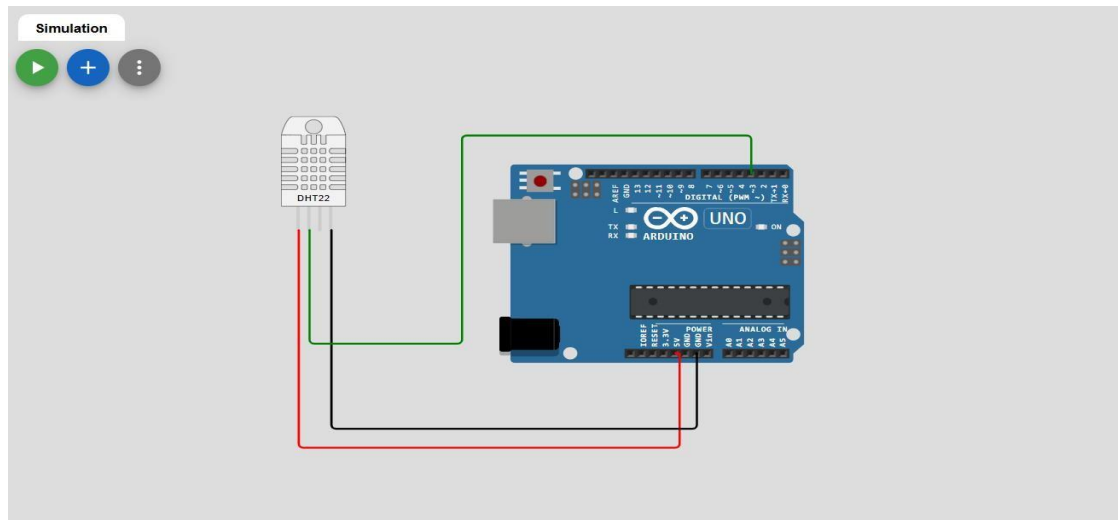
```
Serial.print("Temperature = ");  
Serial.print(temp);  
Serial.print(" °C, ");  
Serial.print("Humidity = ");  
Serial.print(humid);  
Serial.println(" %");  
delay(2000);  
}
```

5. I compiled the program and uploaded it to the Arduino Uno within the Wokwi simulation environment.
6. I opened the **Serial Monitor** and monitored the output. The temperature and humidity readings were displayed continuously at regular intervals.

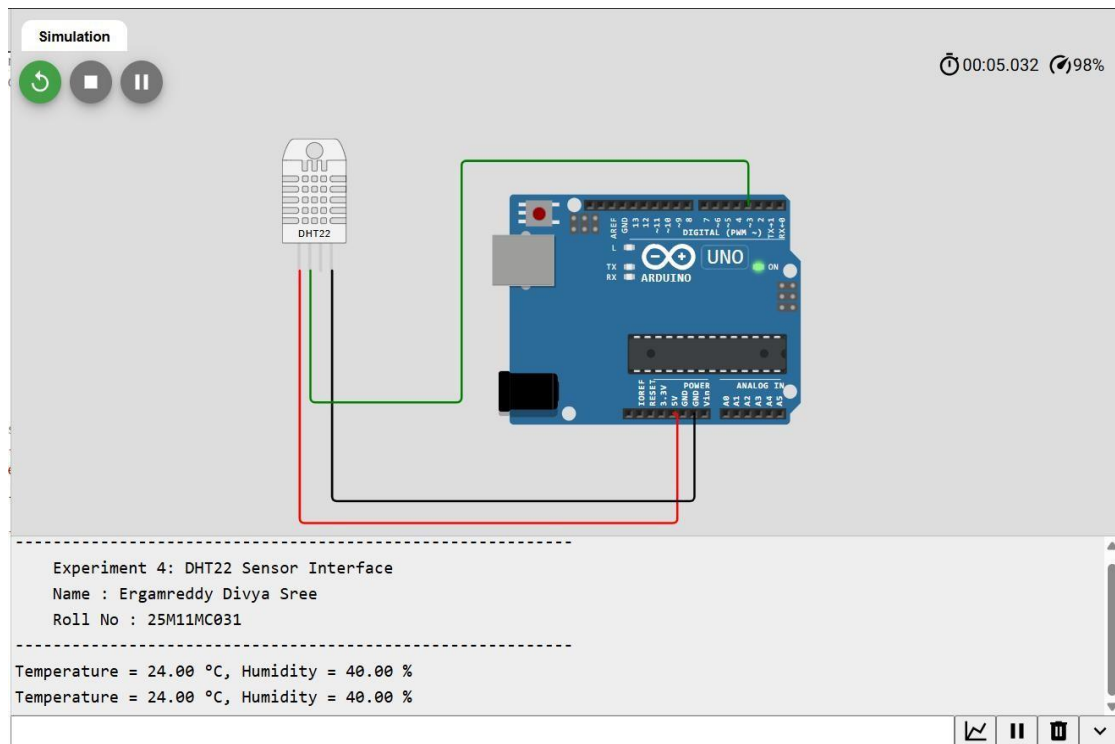
Result:

The DHT22 sensor was successfully interfaced with the Arduino Uno, and the environmental temperature and humidity values were obtained and displayed on the Serial Monitor. The readings were updated continuously during the simulation.

Before:



After:



Conclusion:

This experiment provided practical knowledge of interfacing a digital environmental sensor with an Arduino Uno. I learned how to use the **DHT22 sensor library**, acquire temperature and humidity measurements, and display the collected data through the Serial Monitor. This technique can be applied in IoT-based environmental monitoring, weather stations, and automation systems.

Experiment 5

Aim:

To interface an OLED display with an Arduino Uno and display the real-time temperature and humidity readings obtained from a DHT22 sensor.

Requirements:

- DHT22 Temperature and Humidity Sensor
- Arduino Uno (ATmega328P)
- SSD1306 OLED Display
- Jumper Wires
- Adafruit GFX Library
- Adafruit SSD1306 Library
- DHT Sensor Library

Description:

In this experiment, an SSD1306 OLED display is interfaced with an Arduino Uno along with a DHT22 temperature and humidity sensor. The DHT22 sensor measures the surrounding temperature and humidity, while the Arduino processes the sensor readings and displays them on the OLED screen. The OLED communicates with the Arduino through the I²C communication protocol using the SDA and SCL lines. This experiment demonstrates how sensor data can be collected, processed, and presented on a graphical display, which is an important concept in IoT applications such as smart weather monitoring, environmental monitoring, and home automation.

Procedure:

1. I opened www.wokwi.com in my browser and selected the **Arduino Uno** microcontroller.
2. In the simulation section, I added the required components, including the **DHT22 sensor, SSD1306 OLED display, and jumper wires**, by clicking the “+” symbol.
3. I made the following connections using jumper wires:
 - I connected the **VCC pin of the DHT22** to the **5V power supply** of the Arduino Uno.
 - I connected the **DATA pin of the DHT22** to **digital pin 3** of the Arduino Uno.
 - I connected the **GND pin of the DHT22** to **GND** of the Arduino Uno.
 - I connected **SDA of the OLED** to **A4** and **SCL of the OLED** to **A5**.
 - I connected **VIN of the OLED** to **5V** and **GND to GND** of the Arduino Uno.

4. I installed the required **Adafruit GFX, Adafruit SSD1306, DHT Sensor, and Adafruit Unified Sensor libraries**. I then wrote the required program in sketch.ino to read the sensor values and display them on the OLED.
5. I compiled and uploaded the program to the **Arduino Uno** in the simulation. I then started the simulation.

Code:

/* Experiment 5: Interfacing OLED with Arduino Uno

Purpose: To print temperature and humidity readings on OLED

Name: K.Y.TILAK REDDY

Roll No: 25M11MC077 */

```
#include <Adafruit_Sensor.h>
```

```
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>
```

```
#include <DHT.h>
```

```
#include <DHT_U.h>
```

```
#include <Wire.h>
```

```
// OLED settings
```

```
Adafruit_SSD1306 display(128, 64, &Wire, -1);
```

```
// DHT22 settings
```

```
DHT Sensor(3, DHT22);
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    Sensor.begin();
```

```
    // Initialize OLED
```

```
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
```

```
        Serial.println("SSD1306 allocation failed");
```

```
        for (;;);
```

```
    }
```

```
    display.clearDisplay();
```

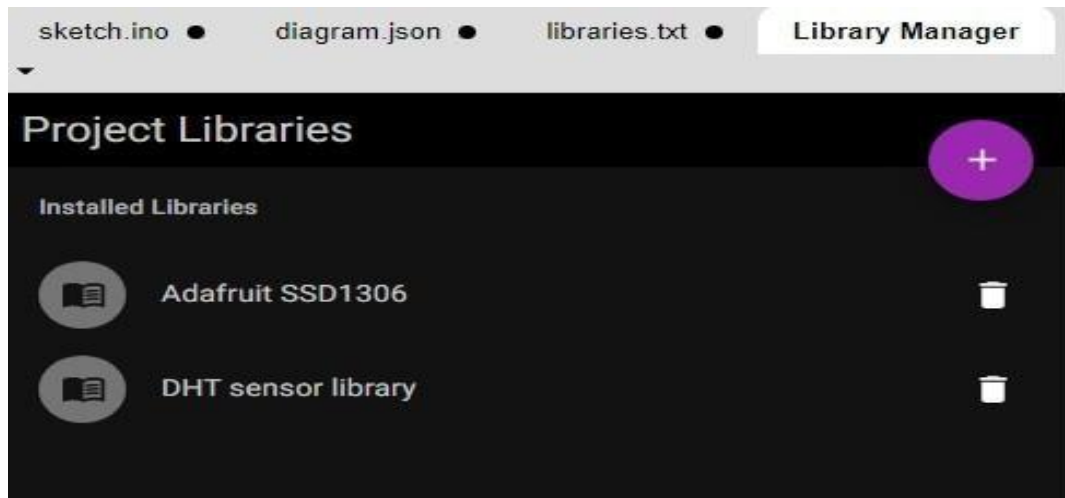
```
    display.setTextColor(WHITE);
```

```
    display.setTextSize(1);
```

```
// Show student details
display.setCursor(0, 10);
display.println("Experiment 5");
display.println("OLED with DHT22");
display.println(" ----- ");
display.println("Name:");
display.println("K.Y.TILAK REDDY");
display.println("Roll No: 25M11MC077");
display.display();
delay(3000);
display.clearDisplay();
}

void loop() {
    // Read temperature and humidity
    float tempr = Sensor.readTemperature();
    float humid = Sensor.readHumidity();
    // Display sensor readings
    display.clearDisplay();
    display.setTextSize(1);
    display.setCursor(0, 10);
    display.print("Temp: ");
    display.print(tempr);
    display.println(" C");
    display.print("Humidity: ");
    display.print(humid);
    display.println(" %");
    display.display();
    delay(1000); // Update every second
}
```

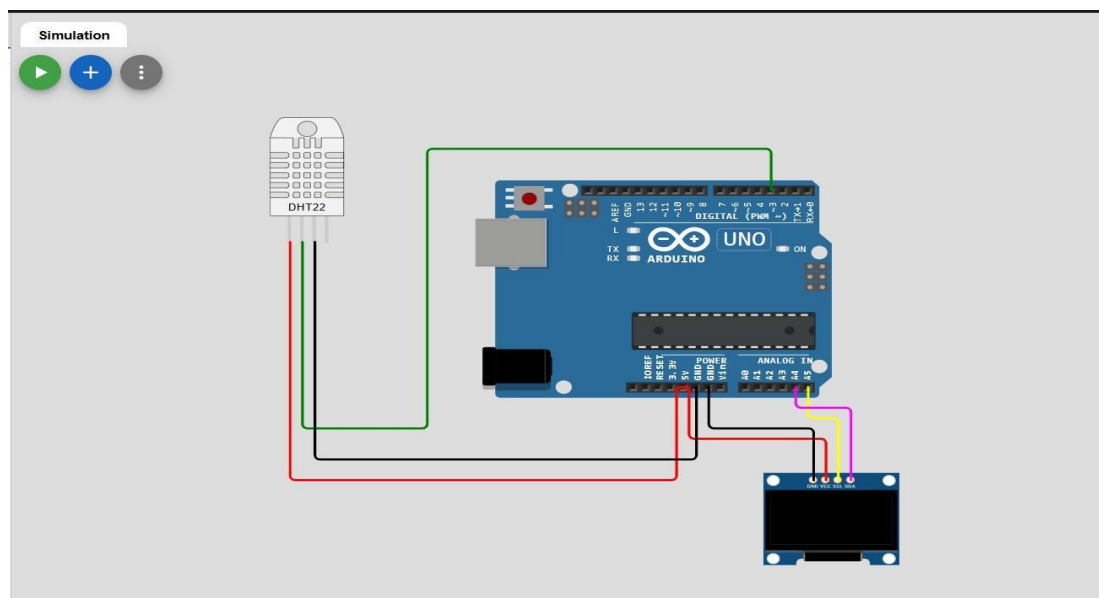

6. I observed the **OLED display** and verified that the **temperature and humidity values** from the DHT22 sensor were displayed and updated continuously.



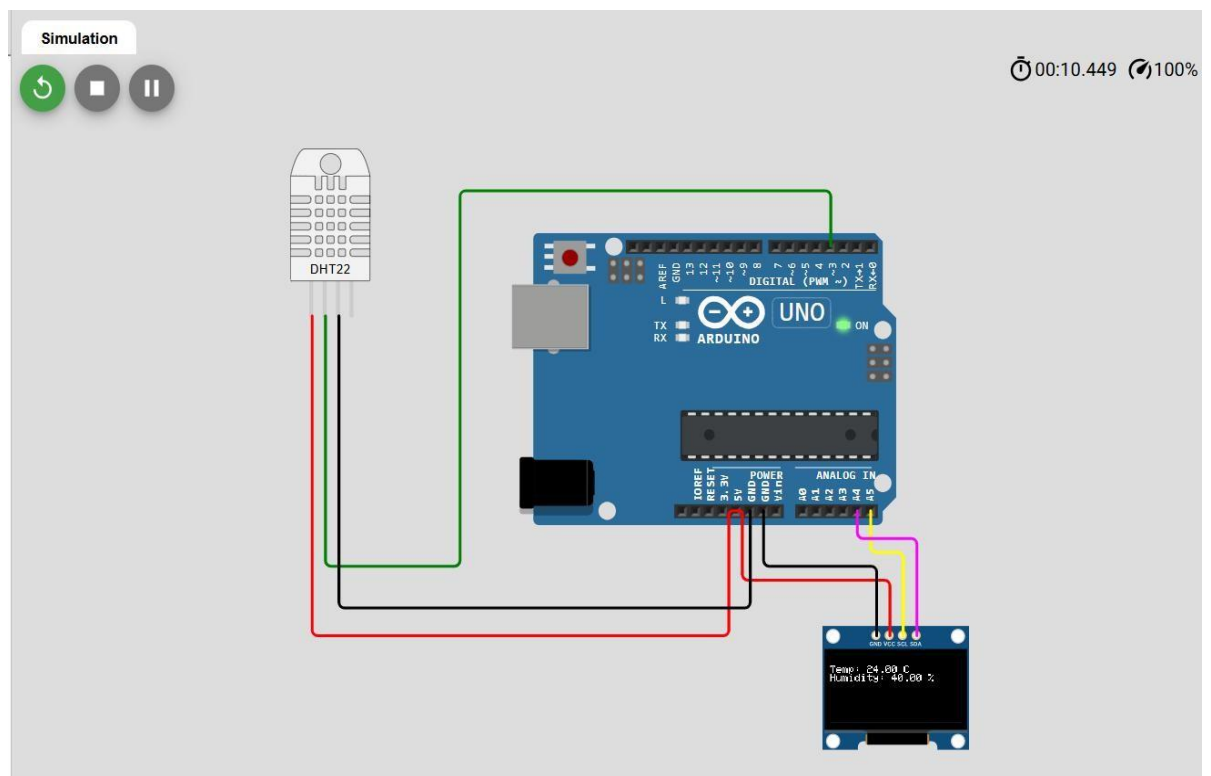
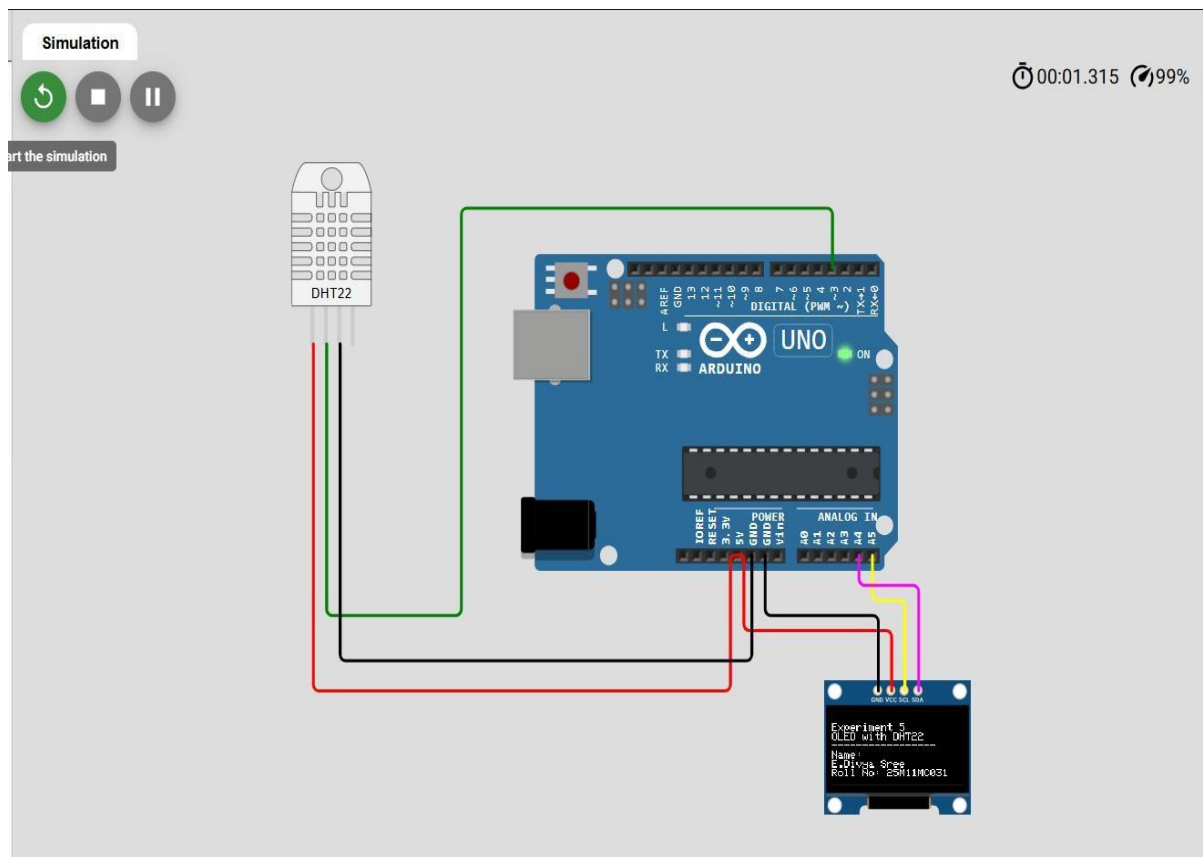
Result:

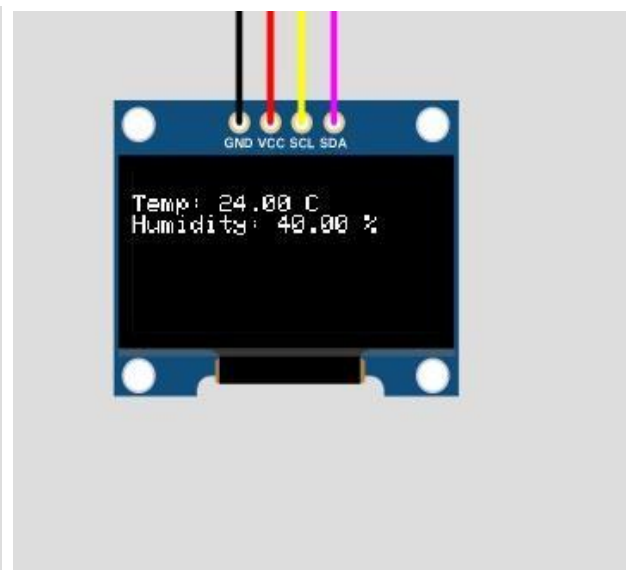
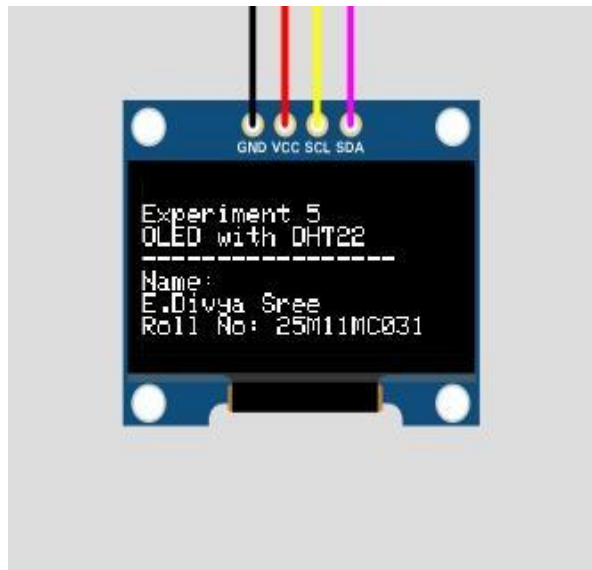
I successfully interfaced the OLED display with the Arduino Uno and observed that the real-time temperature and humidity readings from the DHT22 sensor were displayed on the OLED screen.

Before:



After:





Conclusion:

This experiment provided practical knowledge of interfacing an OLED display with an Arduino Uno. I learned how to use the **I²C protocol** for OLED communication. I also learned how to read sensor data and display it in real time. This concept is useful in IoT applications such as weather monitoring and home automation.